



DR B. DIOP

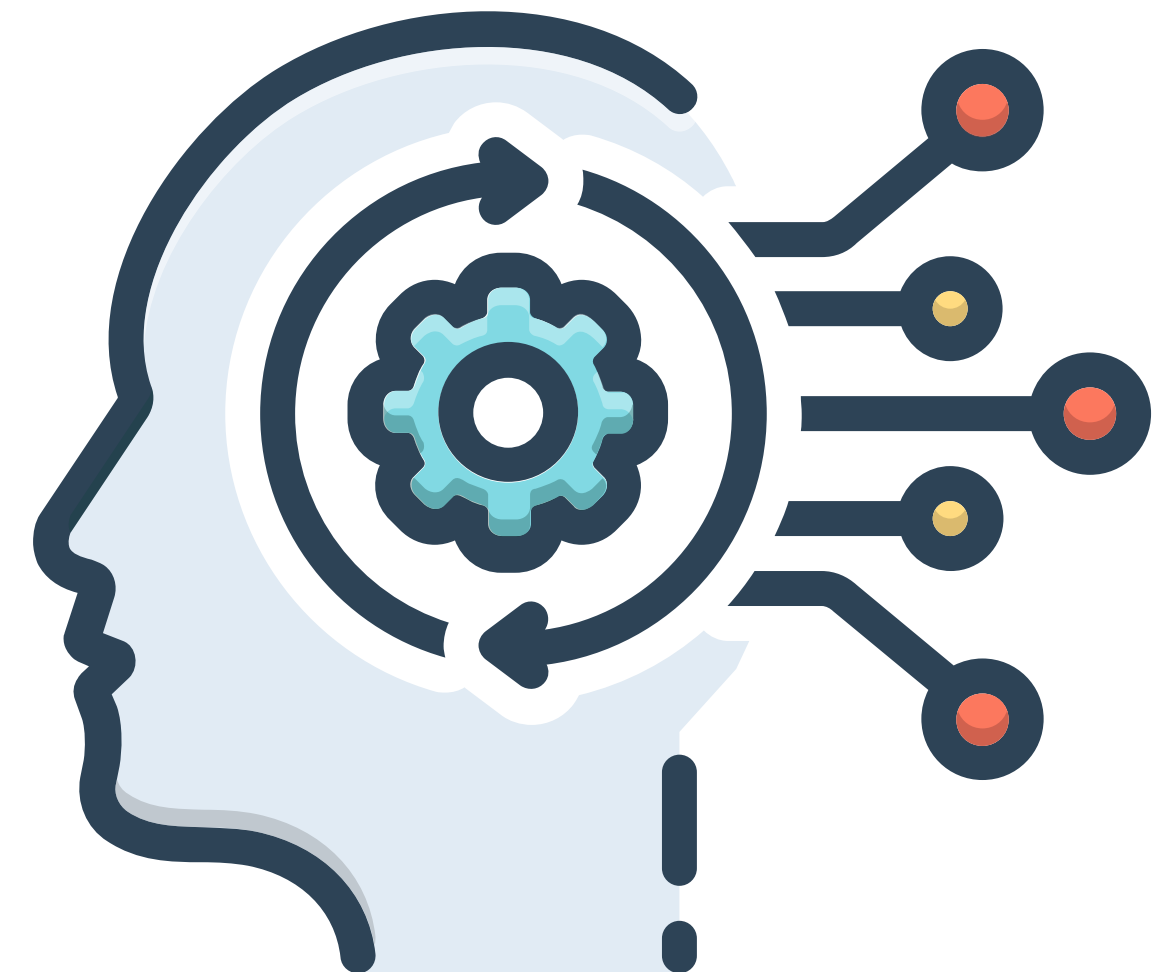
babacar.diop@ussein.edu.sn

Algorithmes et Programmation II

Algorithmes de tri en C

INFO – AGROTIC

Last update: january 2026



Trier = Organiser = Trouver + Comprendre + Analyser + Décider

💡 PENSEZ-Y :

- **Votre playlist Spotify** → triée par artiste/album
- **Votre fil Instagram** → trié par date
- **Vos fichiers** → triés par nom/taille/type
- **Vos notes** → triées par matière/date

Le tri transforme le chaos en ordre ! ✨

Sans Tri	Avec Tri
Recherche lente 🔍	Recherche rapide ⚡
Données désorganisées 🌀	Données structurées 📁
Difficile à analyser 🤖	Facile à comprendre 😊
Inefficace pour l'ordinateur 🐌	Optimisé pour les traitements 🚀



Vue d'Ensemble et Complexités en Informatique

Tableau Comparatif des Algorithmes de Tri

Algorithme	Meilleur cas	Cas moyen	Pire cas	Espace	Stable	Type
Tri à bulles	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	✓	Comparaison
Tri par sélection	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	✗	Comparaison
Tri par insertion	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	✓	Comparaison
Tri fusion	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	✓	Diviser pour régner
Tri rapide	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	✗	Diviser pour régner
Tri par tas	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$	✗	Comparaison
Tri par comptage	$O(n+k)$	$O(n+k)$	$O(n+k)$	$O(k)$	✓	Non-comparaison

Légende :

n = nombre d'éléments
 k = plage des valeurs

Définitions Importantes

Stabilité : Un tri est stable si deux éléments égaux conservent leur ordre relatif initial.

Tri en place : Utilise $O(1)$ mémoire additionnelle (pas de tableau auxiliaire de taille n).

Tri par comparaison : Borne inférieure théorique de $\Omega(n \log n)$ dans le cas moyen.

Tri simples

Tris Simples ($O(n^2)$) : tri à bulles (1/2)

1. Tri à Bulles (Bubble Sort)

Principe : Compare les éléments adjacents et les échange si nécessaire. Les plus grands "remontent" comme des bulles.

```
void tri_bulles(int tab[], int n) {  
    for (int i = 0; i < n - 1; i++) {  
        int echange = 0;  
        for (int j = 0; j < n - i - 1; j++) {  
            if (tab[j] > tab[j + 1]) {  
                // Échange  
                int temp = tab[j];  
                tab[j] = tab[j + 1];  
                tab[j + 1] = temp;  
                echange = 1;  
            }  
        }  
        if (!echange) break; // Optimisation : déjà trié  
    }  
}
```

Tris Simples ($O(n^2)$) : tri à bulles (2/2)

1. Tri à Bulles (Bubble Sort)

Pass 1: [2, 5, 1, 8, 9] (9 en position finale)

Pass 2: [2, 1, 5, 8, 9] (8 en position finale)

Pass 3: [1, 2, 5, 8, 9] (5 en position finale)

Pass 4: [1, 2, 5, 8, 9] (déjà trié)

Tris Simples ($O(n^2)$) : tri par sélection

2. Tri par Sélection (Selection Sort)

Principe : Cherche le minimum dans la partie non triée et le place au début.

Avantage : Très efficace sur des données presque triées ($O(n)$ dans le meilleur cas).

```
void tri_selection(int tab[], int n) {  
    for (int i = 0; i < n - 1; i++) {  
        int min_idx = i;  
        // Trouver le minimum dans [i+1, n-1]  
        for (int j = i + 1; j < n; j++) {  
            if (tab[j] < tab[min_idx]) {  
                min_idx = j;  
            }  
        }  
        // Échanger  
        int temp = tab[i];  
        tab[i] = tab[min_idx];  
        tab[min_idx] = temp;  
    }  
}
```


Tris Simples ($O(n^2)$) : tri par insertion

Tri par Insertion (Insertion Sort)

Principe : Insère chaque élément à sa place dans la partie déjà triée (comme trier des cartes).

Avantage : Très efficace sur des données presque triées ($O(n)$ dans le meilleur cas).

```
void tri_insertion(int tab[], int n) {  
    for (int i = 1; i < n; i++) {  
        int cle = tab[i];  
        int j = i - 1;  
  
        // Décaler les éléments plus grands que cle  
        while (j >= 0 && tab[j] > cle) {  
            tab[j + 1] = tab[j];  
            j--;  
        }  
        tab[j + 1] = cle;  
    }  
}
```

Tri par fusion

Tri Fusion (Merge Sort) : algorithme (1/3)

Principe : Diviser pour Régner

1. **Diviser** : Séparer le tableau en deux moitiés
2. **Régner** : Trier récursivement chaque moitié
3. **Combiner** : Fusionner les deux moitiés triées

```
void fusion(int tab[], int gauche, int milieu, int droite) {  
    int n1 = milieu - gauche + 1;  
    int n2 = droite - milieu;  
  
    // Tableaux temporaires  
    int *G = malloc(n1 * sizeof(int));  
    int *D = malloc(n2 * sizeof(int));  
  
    for (int i = 0; i < n1; i++)  
        G[i] = tab[gauche + i];  
    for (int j = 0; j < n2; j++)  
        D[j] = tab[milieu + 1 + j];  
}
```

Tri Fusion (Merge Sort) : algorithme (2/3)

Principe : Diviser pour Régner

1. **Diviser** : Séparer le tableau en deux moitiés
2. **Régner** : Trier récursivement chaque moitié
3. **Combiner** : Fusionner les deux moitiés triées

// Fusion

```
int i = 0, j = 0, k = gauche;
while (i < n1 && j < n2) {
    if (G[i] <= D[j]) {
        tab[k++] = G[i++];
    } else {
        tab[k++] = D[j++];
    }
}
```

// Copier les éléments restants

```
while (i < n1) tab[k++] = G[i++];
while (j < n2) tab[k++] = D[j++];
```

```
free(G);
```

```
free(D);
```

```
}
```

Tri Fusion (Merge Sort) : algorithme (3/3)

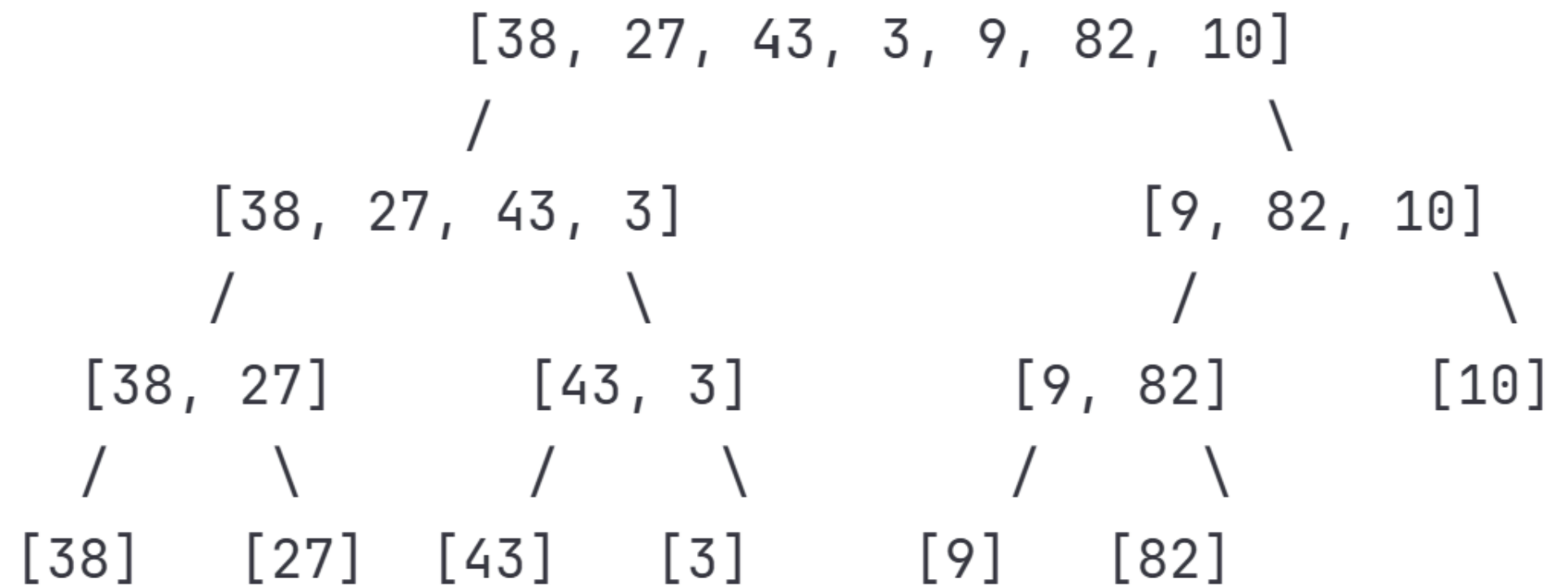
Principe : Diviser pour Régner

1. **Diviser** : Séparer le tableau en deux moitiés
2. **Régner** : Trier récursivement chaque moitié
3. **Combiner** : Fusionner les deux moitiés triées

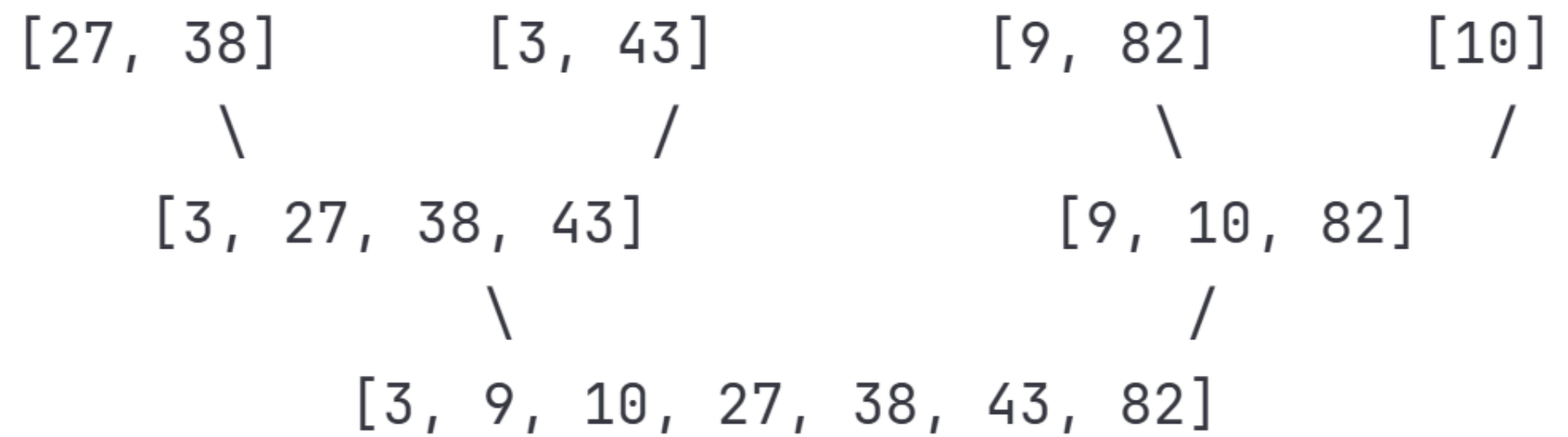
```
void tri_fusion(int tab[], int gauche, int droite) {  
    if (gauche < droite) {  
        int milieu = gauche + (droite - gauche) / 2;  
  
        tri_fusion(tab, gauche, milieu);  
        tri_fusion(tab, milieu + 1, droite);  
        fusion(tab, gauche, milieu, droite);  
    }  
}
```


Tri Fusion (Merge Sort) : arbre de récursion

Pour $[38, 27, 43, 3, 9, 82, 10]$,
voici l'arbre de récursion de
l'algorithme de fusion présenté
précédemment.



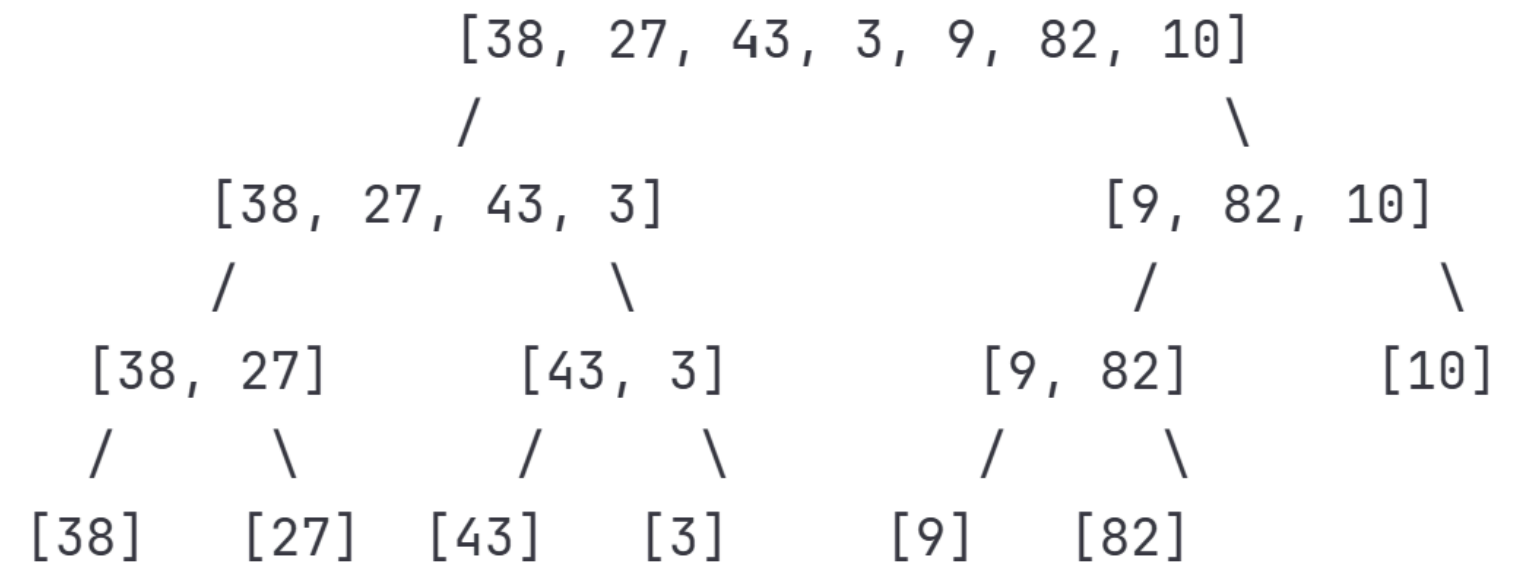
Fusion ascendante :



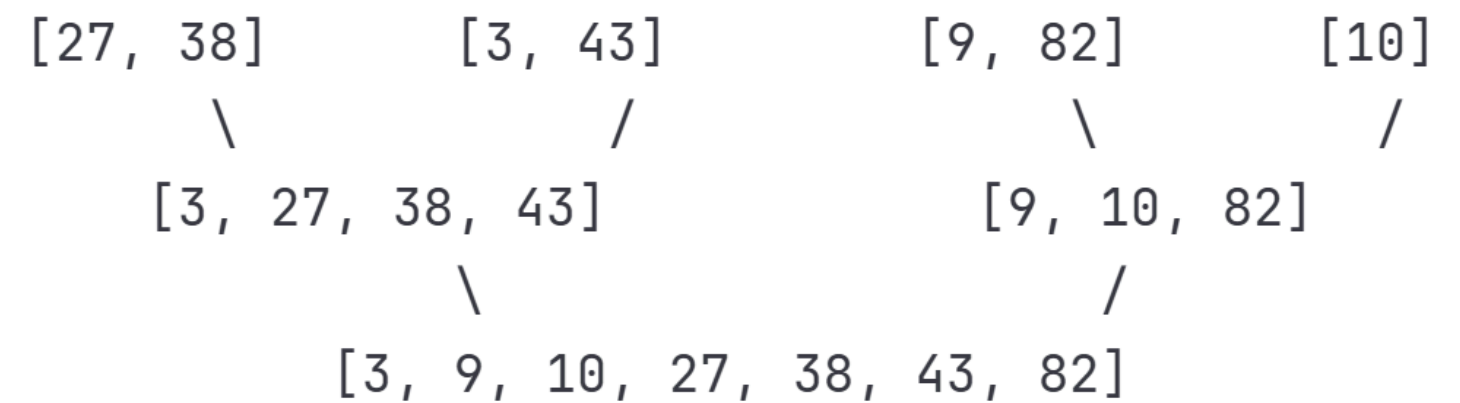
Tri Fusion (Merge Sort) : analyse de complexité

Analyse

- Complexité temporelle :
 - $\Theta(n \log n)$ dans tous les cas
- Complexité spatiale :
 - $O(n)$ pour les tableaux temporaires
- Stable : Oui
- Usage : Idéal pour les grandes données, tri externe



Fusion ascendante :



Tri rapide

Tri Rapide (Quick Sort) : algorithme (1/2)

Principe : Partitionnement

1. Choisir un pivot
2. Partitionner : éléments $<$ pivot à gauche, $>$ pivot à droite
3. Trier récursivement les deux partitions

```
int partitionner(int tab[], int bas, int haut) {  
    int pivot = tab[haut]; // Pivot = dernier élément  
    int i = bas - 1;      // Index du plus petit élément  
  
    for (int j = bas; j < haut; j++) {  
        if (tab[j] < pivot) {  
            i++;  
            // Échanger tab[i] et tab[j]  
            int temp = tab[i];  
            tab[i] = tab[j];  
            tab[j] = temp;  
        }  
    }  
  
    // Placer le pivot à sa position finale  
    int temp = tab[i + 1];  
    tab[i + 1] = tab[haut];  
    tab[haut] = temp;  
  
    return i + 1;  
}
```

Tri Rapide (Quick Sort) : algorithme (2/2)

Principe : Partitionnement

1. Choisir un pivot
2. Partitionner : éléments $<$ pivot à gauche, $>$ pivot à droite
3. Trier récursivement les deux partitions

```
void tri_fusion(int tab[], int gauche, int droite) {  
    if (gauche < droite) {  
        int milieu = gauche + (droite - gauche) / 2;  
  
        tri_fusion(tab, gauche, milieu);  
        tri_fusion(tab, milieu + 1, droite);  
        fusion(tab, gauche, milieu, droite);  
    }  
}
```

Tri Rapide (Quick Sort) : exemple de partitionnement

Tableau initial : $[10, 80, 30, 90, 40, 50, 70]$, pivot = 70

Étape 1: $i=-1, j=0 \rightarrow 10 < 70 \rightarrow i=0$, échange $[10], 80, 30, 90, 40, 50, 70$

Étape 2: $i=0, j=1 \rightarrow 80 > 70 \rightarrow [10, 80], 30, 90, 40, 50, 70$

Étape 3: $i=0, j=2 \rightarrow 30 < 70 \rightarrow i=1$, échange $[10, 30], 80, 90, 40, 50, 70$

Étape 4: $i=1, j=3 \rightarrow 90 > 70 \rightarrow [10, 30, 80], 90, 40, 50, 70$

Étape 5: $i=1, j=4 \rightarrow 40 < 70 \rightarrow i=2$, échange $[10, 30, 40], 90, 80, 50, 70$

Étape 6: $i=2, j=5 \rightarrow 50 < 70 \rightarrow i=3$, échange $[10, 30, 40, 50], 80, 90, 70$

Fin: Placer pivot $\rightarrow [10, 30, 40, 50, 70, 90, 80]$

Choix du Pivot et analyse

Stratégies :

1. Dernier élément (simple mais peut être $O(n^2)$ sur tableau trié)
2. Premier élément
3. Médiane de trois (premier, milieu, dernier) - recommandé
4. Aléatoire - évite le pire cas avec forte probabilité

Analyse

- Meilleur/Moyen cas : $O(n \log n)$
- Pire cas : $O(n^2)$ quand pivot mal choisi (tableau déjà trié)
- Espace : $O(\log n)$ pour la pile de récursion
- En pratique : Souvent le plus rapide grâce à la bonne localité mémoire

Tri par tas

Tri par Tas (Heap Sort) (1/2)

Principe : Structure de Tas

Un tas max est un arbre binaire complet où chaque nœud est $\geq$ à ses enfants.

Représentation en tableau :

Pour l'indice i :

- Parent : $(i-1)/2$
- Enfant gauche : $2*i + 1$
- Enfant droit : $2*i + 2$

```
void tamiser(int tab[], int n, int i) {  
    int max = i;  
    int gauche = 2 * i + 1;  
    int droite = 2 * i + 2;  
  
    // Trouver le plus grand parmi i, gauche, droite  
    if (gauche < n && tab[gauche] > tab[max])  
        max = gauche;  
  
    if (droite < n && tab[droite] > tab[max])  
        max = droite;  
  
    // Si le max n'est pas i, échanger et continuer  
    if (max != i) {  
        int temp = tab[i];  
        tab[i] = tab[max];  
        tab[max] = temp;  
  
        tamiser(tab, n, max);  
    }  
}
```

Tri par Tas (Heap Sort) (2/2)

Principe : Structure de Tas

Un tas max est un arbre binaire complet où chaque nœud est $\geq$ à ses enfants.

Représentation en tableau :

Pour l'indice i :

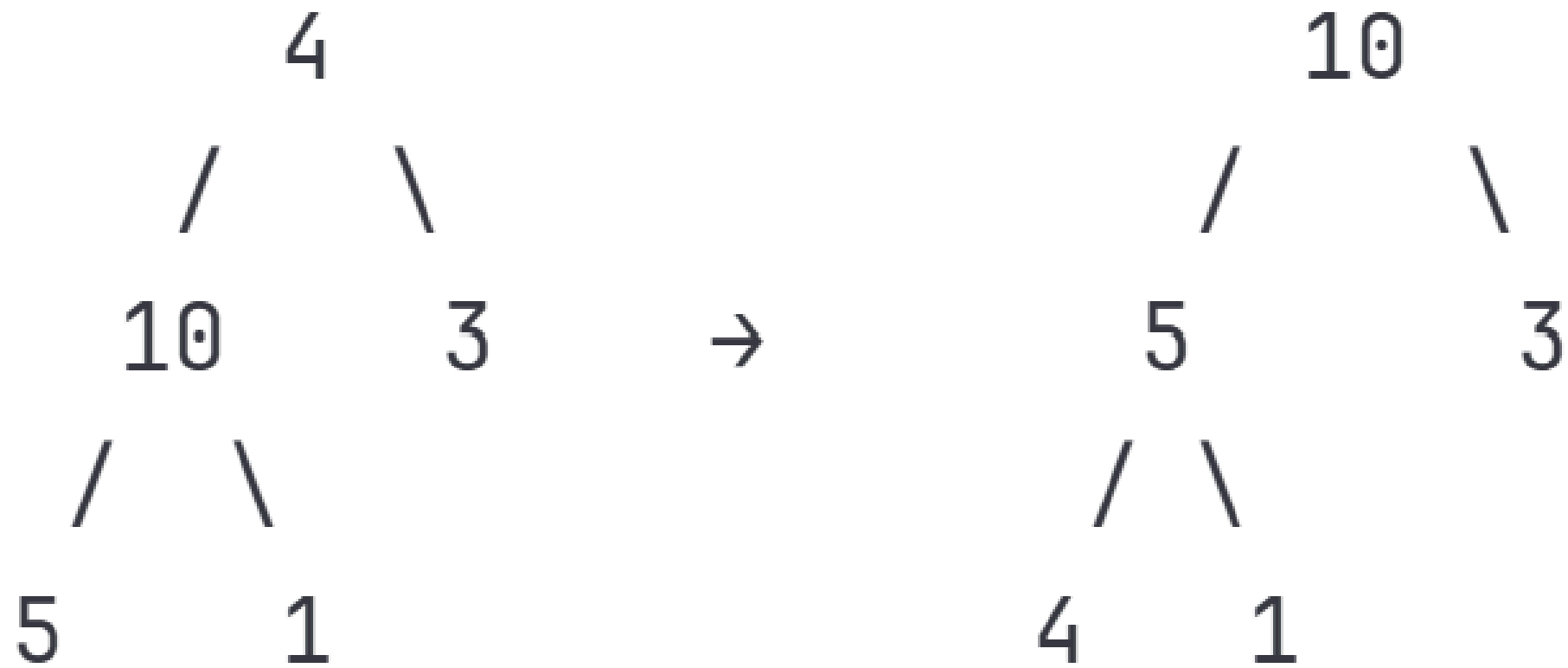
- Parent : $(i-1)/2$
- Enfant gauche : $2*i + 1$
- Enfant droit : $2*i + 2$

```
void tri_tas(int tab[], int n) {  
    // 1. Construire le tas max  
    for (int i = n / 2 - 1; i >= 0; i--)  
        tamiser(tab, n, i);  
  
    // 2. Extraire les éléments un par un  
    for (int i = n - 1; i > 0; i--) {  
        // Déplacer la racine (max) à la fin  
        int temp = tab[0];  
        tab[0] = tab[i];  
        tab[i] = temp;  
  
        // Tamiser sur le tas réduit  
        tamiser(tab, i, 0);  
    }  
}
```

Tri par Tas (Heap Sort) : visualisation (1/3)

Tableau : [4, 10, 3, 5, 1]

Étape 1 : Construction du tas max



Tri par Tas (Heap Sort) : visualisation (2/3)

Tableau : [4, 10, 3, 5, 1]

Étape 2 : Extraction et tri

Extraction 10 → [1, 5, 3, 4 | 10]

Tamiser → [5, 4, 3, 1 | 10]

Extraction 5 → [1, 4, 3 | 5, 10]

Tamiser → [4, 1, 3 | 5, 10]

Extraction 4 → [3, 1 | 4, 5, 10]

Tamiser → [3, 1 | 4, 5, 10]

Extraction 3 → [1 | 3, 4, 5, 10]

Résultat : [1, 3, 4, 5, 10]

Tri par Tas (Heap Sort) : visualisation (3/3)

Tableau : [4, 10, 3, 5, 1]

Étape 2 : Extraction et tri

Extraction 10 → [1, 5, 3, 4 | 10]

Tamiser → [5, 4, 3, 1 | 10]

Extraction 5 → [1, 4, 3 | 5, 10]

Tamiser → [4, 1, 3 | 5, 10]

Extraction 4 → [3, 1 | 4, 5, 10]

Tamiser → [3, 1 | 4, 5, 10]

Extraction 3 → [1 | 3, 4, 5, 10]

Résultat : [1, 3, 4, 5, 10]

Analyse

- **Complexité** : $O(n \log n)$ dans tous les cas
- **Espace** : $O(1)$ - tri en place
- **Pas stable**
- **Usage** : Quand on veut garantir $O(n \log n)$ avec $O(1)$ mémoire

Tri par comptage

Tri par Comptage (Counting Sort) (1/4)

Principe : Tri Non-Comparatif

Compte les occurrences de chaque valeur.

Hypothèse : valeurs dans $[0, k]$.

```
void tri_comptage(int tab[], int n, int k) {  
    // k = valeur maximale dans le tableau  
  
    // Tableau de comptage  
    int *comptage = calloc(k + 1, sizeof(int));  
    int *sortie = malloc(n * sizeof(int));  
  
    // Compter les occurrences  
    for (int i = 0; i < n; i++)  
        comptage[tab[i]]++;  
  
    // Transformer en positions cumulatives  
    for (int i = 1; i <= k; i++)  
        comptage[i] += comptage[i - 1];  
  
    // Construire le tableau trié (de droite à gauche pour stabilité)  
    for (int i = n - 1; i >= 0; i--) {  
        sortie[comptage[tab[i]] - 1] = tab[i];  
        comptage[tab[i]]--;  
    }  
  
    // Copier dans le tableau original  
    for (int i = 0; i < n; i++)  
        tab[i] = sortie[i];  
  
    free(comptage);  
    free(sortie);  
}
```

Tri par Comptage (Counting Sort) (2/4)

Exemple Détaillé

Tableau : $[2, 5, 3, 0, 2, 3, 0, 3]$, $k = 5$

Étape 1 : Comptage

Valeur :	0	1	2	3	4	5
Compte :	2	0	2	3	0	1

Tri par Comptage (Counting Sort) (3/4)

Exemple Détaillé

Tableau : $[2, 5, 3, 0, 2, 3, 0, 3]$, $k = 5$

Étape 2 : Positions cumulatives

Valeur :	0	1	2	3	4	5
Cumul :	2	2	4	7	7	8

Tri par Comptage (Counting Sort) (4/4)

Exemple Détaillé

Tableau : $[2, 5, 3, 0, 2, 3, 0, 3]$, $k = 5$

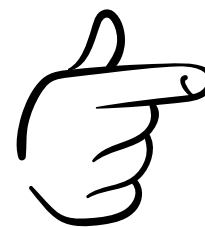
Étape 3 : Construction (lecture de droite à gauche)

Lire 3 $\rightarrow$ position 7 $\rightarrow$ sortie[6] = 3, cumul[3] = 6

Lire 0 $\rightarrow$ position 2 $\rightarrow$ sortie[1] = 0, cumul[0] = 1

Lire 3 $\rightarrow$ position 6 $\rightarrow$ sortie[5] = 3, cumul[3] = 5

...

 **Résultat :** $[0, 0, 2, 2, 3, 3, 3, 5]$

Tri par Comptage (Counting Sort) : Analyse

- **Complexité** : $O(n + k)$
- **Espace** : $O(n + k)$
- **Stable** : Oui
- **Efficace quand** : $k = O(n)$, c'est-à-dire plage des valeurs pas trop grande
- **Usage** : Tri de caractères, petits entiers, comme sous-routine du tri radix

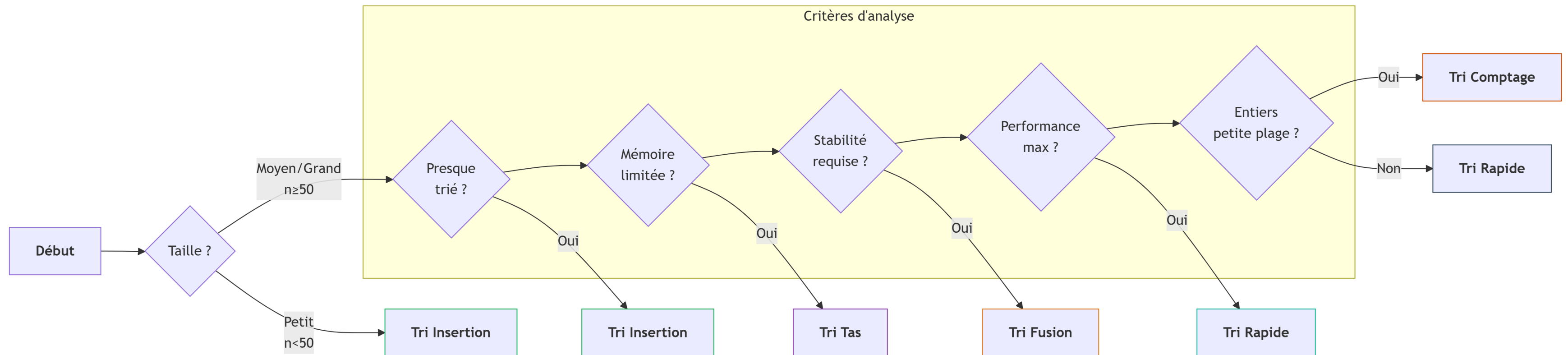
Comparaison Empirique (Temps d'exécution relatif)

Pour $n = 10\,000$ éléments aléatoires :

Tri à bulles	:		(100x)
Tri par sélection	:		(100x)
Tri par insertion	:		(50x)
Tri fusion	:		(10x)
Tri rapide	:		(5x) ← Plus rapide en pratique
Tri par tas	:		(8x)
Tri par comptage*	:		(3x) *si k petit

Guide de Choix d'Algorithme de tri adéquat

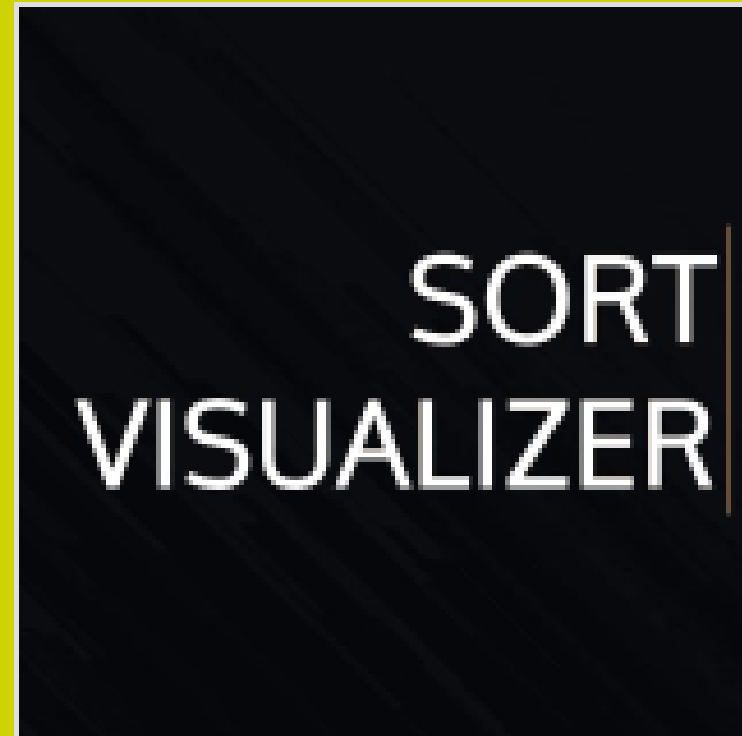
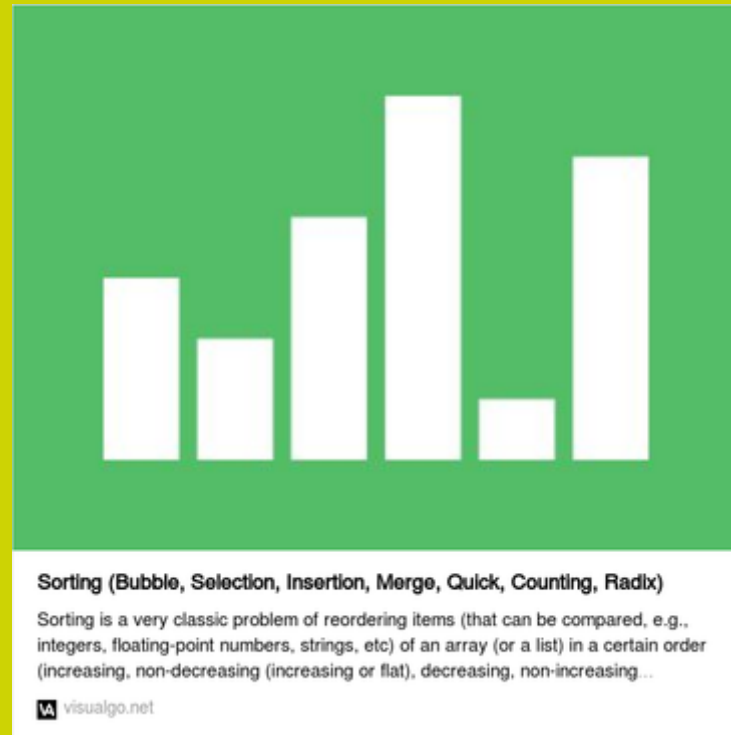
Arbre de Décision



Points Clés à Retenir

1. **Pas d'algorithme universel** : choisir selon le contexte
2. **Tris simples ($O(n^2)$)** : valables pour petites données
3. **Tris avancés ($O(n \log n)$)** : nécessaires pour grandes données
4. **Tri rapide** : souvent le plus rapide en pratique
5. **Tri fusion** : choix sûr si stabilité ou garantie $O(n \log n)$ requise
6. **Tri par comptage** : exceptionnel pour entiers dans petite plage
7. **Toujours analyser** : taille, distribution, contraintes mémoire/stabilité

Outils de visualisation ONLINE !!



Sort Visualizer

A visualization of 15+ sorting algorithms, including Quick Sort, Merge Sort, Selection Sort and more!

 sortvisualizer.com